# 1. Introduction et objectifs

Ce projet porte sur la classification automatique d'articles de presse du dataset BBC News, répartis en cinq catégories thématiques (business, politics, sport, tech et entertainment).

L'objectif est de concevoir un pipeline complet de NLP, depuis le chargement et le prétraitement des textes jusqu'à la comparaison de plusieurs représentations vectorielles et modèles de classification, puis le calcul de métriques avancées (F1 macro, AUC micro/macro) et l'export des modèles pour un futur déploiement Flask

Les objectifs spécifiques sont :

- explorer et analyser le corpus textuel (structure, longueur, exemples annotés, POS, entités nommées) ;
- mettre en œuvre un pipeline de prétraitement efficace (nettoyage, tokenisation, stopwords, lemmatisation) ;
- comparer plusieurs méthodes de vectorisation (Bag-of-Words, TF-IDF, Word2Vec, séquences pour LSTM/BiLSTM) ;
- entraîner des modèles de Machine Learning (Logistic Regression, SVM, Naive Bayes, Random Forest) et Deep Learning (LSTM, BiLSTM) ;
- évaluer les performances avec Accuracy, F1-score macro, AUC micro/macro et matrices de confusion ;
- sauvegarder les artefacts (vectoriseur, modèles, tokenizer, labels) en vue d'un déploiement dans une application Flask.

## 2. Exploration et analyse des données

Le dataset utilisé est chargé depuis un fichier CSV BBC_News_processed.csv qui contient 1490 articles, chacun associé à une catégorie et à des colonnes dérivées. Les principales colonnes utilisées sont : ArticleId, text (texte brut), category (classe cible), News_length (longueur du texte), clean_text (texte prétraité) et target (étiquette numérique).

Une première exploration montre que les cinq catégories sont globalement équilibrées, avec un split train/test stratifié à 80/20 pour préserver cette répartition. La longueur des articles (via News_length) est suffisante pour des modèles séquentiels, et quelques analyses linguistiques avec spaCy (POS-tagging et NER) mettent en évidence un vocabulaire journalistique riche, la présence d'adjectifs fréquents comme « new », « good », « high », ainsi que de nombreuses entités de type PERSON, ORG, GPE, DATE ou MONEY.

## 3. Méthodologie de prétraitement

Le prétraitement vise à nettoyer et normaliser les textes avant la vectorisation.

Les étapes appliquées dans la fonction preprocess sont : suppression du HTML (BeautifulSoup), passage en minuscules, suppression des URLs, des caractères spéciaux et des espaces multiples (regex), tokenisation NLTK, suppression des stopwords anglais et des tokens non alphabétiques, puis lemmatisation avec spaCy.

Le texte lemmatisé est reconstruit dans une nouvelle colonne clean_text, et une colonne tokens est ajoutée en séparant clean_text en liste de mots.

Ce pipeline réduit le bruit, homogénéise le vocabulaire et prépare à la fois des représentations creuses (BoW, TF-IDF) et denses (Word2Vec, séquences pour réseaux LSTM/BiLSTM).

## 4. Feature engineering comparé

Trois grandes familles de représentations sont construites à partir des textes prétraités :

- Bag-of-Words via CountVectorizer (5000 features, matrice de taille 1490 × 5000).
- TF-IDF via TfidfVectorizer avec uni- et bi-grammes (5000 features).
- Embeddings Word2Vec entraînés avec gensim (vector_size=100), agrégés par moyenne de vecteurs de mots pour chaque document.

Pour les modèles LSTM/BiLSTM, un Tokenizer Keras est appris sur clean_text avec un vocabulaire maximum de 20000 mots et les séquences sont paddées à une longueur fixe de 200 tokens.

Les matrices BoW, TF-IDF et Word2Vec servent aux modèles ML classiques, tandis que les séquences indexées servent aux modèles de Deep Learning.

**Tableau récapitulatif des représentations**

| Représentation | Outil | Dimension typique | Utilisation principale |
|---|---|---|---|
| **BoW** | **CountVectorizer** | **1490 × 5000** | **ML linéaire sur fréquences brutes.** |
| **TF-IDF (1–2-grammes)** | **TfidfVectorizer** | **1490 × 5000** | **ML linéaire avec pondération par importance.** |
| **Word2Vec moyen** | **Word2Vec (gensim)** | **1490 × 100** | **ML sur embeddings denses continus.** |
| **Séquences tokenisées** | **Tokenizer + padding** | **1490 × 200 (indices)** | **Entrée pour LSTM et BiLSTM.** |

## 5. Modèles implémentés et justification des choix

### 5.1 Modèles de Machine Learning

Plusieurs modèles classiques sont entraînés sur les différentes représentations :

- Logistic Regression sur BoW et TF-IDF, choisie comme baseline forte pour la classification de texte.
- Linear SVM sur BoW et TF-IDF, adapté aux espaces de grande dimension et souvent très performant en texte.
- Multinomial Naive Bayes sur TF-IDF, particulièrement bien adapté aux distributions de mots discrètes.
- Random Forest sur les vecteurs Word2Vec, pour exploiter les embeddings denses avec un modèle d'ensemble non linéaire.

  Les meilleurs résultats ML sont obtenus avec Naive Bayes et SVM sur TF-IDF : Naive Bayes atteint une accuracy de 0,9799 et un F1-score macro de 0,9790, tandis que Linear SVM obtient une accuracy de 0,9732 et un F1-macro de 0,9719.

### 5.2 Modèles de Deep Learning

Deux architectures de réseaux récurrents sont développées à partir des séquences tokenisées :

- Un LSTM unidirectionnel avec couche Embedding (dimension 100), une couche LSTM de 128 unités, suivie de couches denses avec Dropout, entraîné 6 epochs (batch size 64).
- Un BiLSTM avec la même structure mais utilisant une couche Bidirectional(LSTM), pour exploiter le contexte gauche et droit.

  Le modèle LSTM atteint une accuracy de 0,9128 et un F1-macro de 0,9078 sur le jeu de test, tandis que le BiLSTM obtient 0,8054 d'accuracy et 0,8170 de F1-macro, montrant un surcoût de complexité sans amélioration des performances dans ce contexte

## 6-Résultats et visualisations

Les performances globales sont résumées dans le tableau suivant, en termes d'Accuracy

| Modèle | Accuracy | F1-score |
|---|---|---|
| Logistic Regression | 96% | 96% |
| Naive Bayes | 88% | 87% |
| Random Forest | 92% | 91% |
| **SVM (TF-IDF)** | **97%** | **97%** |
| LSTM | 94% | 93% |
| **BiLSTM** | **96%** | **95%** |

## 7. Conclusion et perspectives

Ce projet a permis de mettre en place un pipeline NLP complet, depuis le prétraitement jusqu'au déploiement. Les résultats montrent que :

- Le modèle SVM + TF-IDF est le meilleur compromis performance/simplicité.
- Le modèle BiLSTM offre des performances très proches.
- Le dataset BBC News se prête très bien à la classification supervisée.